

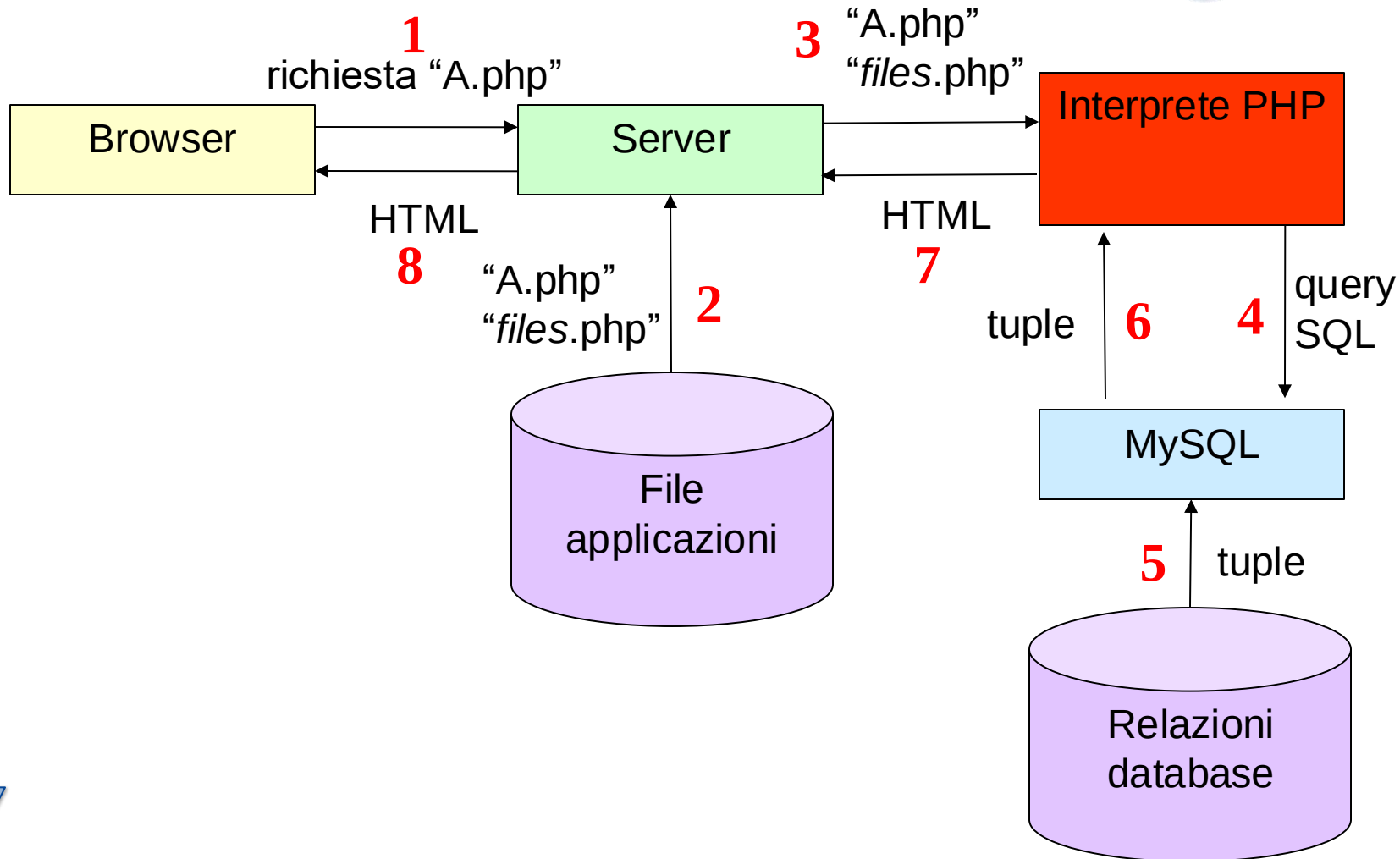


PROGRAMMAZIONE WEB E SERVIZI DIGITALI

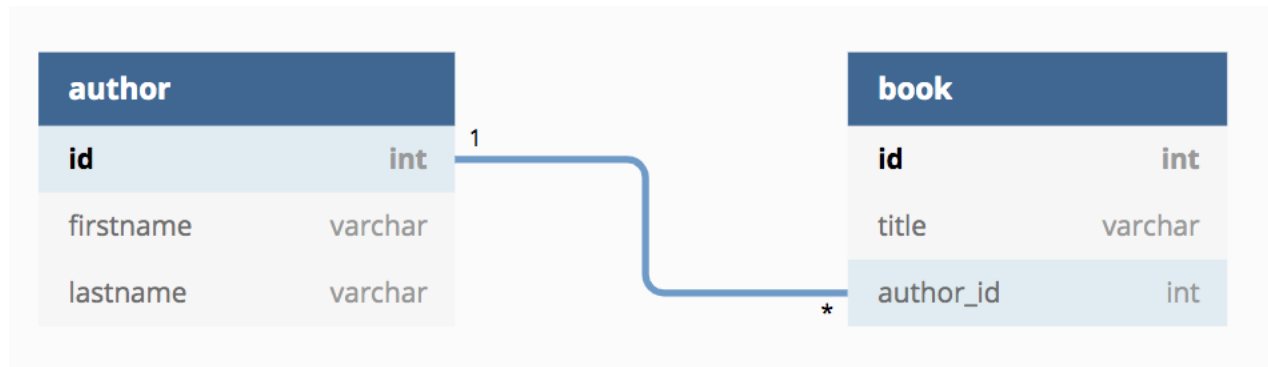
PHP E STRUMENTI DI PERSISTENZA (MySQL)



Interazione tra PHP e MySQL



Il database



Connessione a MySQL



```
new PDO("mysql:host={$HOST};dbname={$DB_NAME}", $USERNAME, $PASSWORD);
```

PHP Data Object - apre una connessione al server ***\$HOST***, database ***\$DB_NAME***, con utente ***\$USERNAME*** e password ***\$PASSWORD***



Esempio (I)



```
<?php
$USERNAME = "devis";
$PASSWORD = "bianchini";
$HOST = "localhost";
$DB_NAME = "MyLibraryDB";
```

utils/config.php



Esempio (II)



```
include_once('../utils/config.php');  
include_once('Author.php');  
include_once('Book.php');
```

2 references | 0 implementations

class DataLayer



4 references

private \$pdo;

/**

*** Constructor of the DataLayer class**

***/**

2 references | 0 overrides

public function __construct()

{

global \$HOST, \$USERNAME, \$PASSWORD, \$DB_NAME;

try {

 \$this->pdo = new PDO("mysql:host={\$HOST};dbname={\$DB_NAME}", \$USERNAME, \$PASSWORD);

 // Set the PDO error mode to exception

 \$this->pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

} catch(PDOException \$e){

 die("ERROR: Could not connect. " . \$e->getMessage());

}

}



Esecuzione di query SQL



(new PDO) ->query(\$sql):

invia una query SQL al database a cui si è attualmente connessi

a)

```
$sql = "SELECT * FROM book ORDER BY title";  
try {  
    $statement = $this->pdo->query($sql);  
}
```

b)

```
$sql = "SELECT * FROM author where id = :id";  
try {  
    $statement = $this->pdo->prepare($sql);  
    $statement->execute(array(':id' => $id));  
}
```

c)

```
$sql = "SELECT * FROM author where id = :id";  
try {  
    $statement = $this->pdo->prepare($sql);  
    $statement->bindParam(':id', $id);  
    $statement->execute();  
}
```

Elaborazione del risultato (I)



`$queryStatement->fetch(PDO::FETCH_ASSOC)` :
estrae un record dal risultato di una query (ogni volta che viene invocata) sotto forma di array associativo (i nomi delle colonne del record diventano i nomi degli elementi dell' array)

```
public function findAuthorById($id)
{
    $sql = "SELECT * FROM author where id = :id";
    try {
        $statement = $this->pdo->prepare($sql);
        $statement->execute(array(':id' => $id));

        $author = $statement->fetch(PDO::FETCH_ASSOC);

        // Check if author found
        if ($author) {
            return new Author($author['id'], $author['firstname'], $author['lastname']);
        } else {
            return null; // Author not found
        }
    } catch (PDOException $e) {
        die("ERROR: Could not execute query. " . $e->getMessage());
    }
}
```


Elaborazione del risultato (I)



`$queryStatement->fetch(PDO::FETCH_ASSOC)` :
estrae un record dal risultato di una query (ogni volta che viene invocata) sotto forma di array associativo (i nomi delle colonne del record diventano i nomi degli elementi dell' array)

```
public function findAuthorById($id)
{
    $sql = "SELECT * FROM author where id = :id";
    try {
        $statement = $this->pdo->prepare($sql);
        $statement->execute(array(':id' => $id));

        $author = $statement->fetch(PDO::FETCH_ASSOC);

        // Check if author found
        if ($author) {
            return new Author($author['id'], $author['firstname'], $author['lastname']);
        } else {
            return null; // Author not found
        }
    } catch (PDOException $e) {
        die("ERROR: Could not execute query. " . $e->getMessage());
    }
}
```

`$author`

id	2
firstname	Alessandro
lastname	Manzoni

Elaborazione del risultato (II)



`$queryStatement->fetchAll(PDO::FETCH_ASSOC)`:
simile al precedente, ma in questo caso recupera tutti i record nel risultato di una query, mettendoli in un array di array associativi

```
public function listBooks()
{
    $sql = "SELECT * FROM book ORDER BY title";
    try {
        $statement = $this->pdo->query($sql);

        $books = $statement->fetchAll(PDO::FETCH_ASSOC);
        $booksList = array();
        foreach ($books as $book) {
            $author = $this->findAuthorById($book['author_id']);
            $booksList[] = new Book($book['id'], $book['title'], $author->getFirstName() . " " . $author->getLastName(), $book['author_id']);
        }
        return $booksList;
    } catch (PDOException $e) {
        die("ERROR: Could not execute query. " . $e->getMessage());
    }
}
```

Modifica del database



`$queryStatement->rowCount()` :

restituisce il numero di tuple coinvolte nella query appena eseguita

```
public function findBooksByAuthorID($author_id)
{
    $sql = "SELECT * FROM book where author_id = :author_id";
    try {
        $statement = $this->pdo->prepare($sql);
        $statement->bindParam(':author_id', $author_id);
        $statement->execute();

        // Get the number of affected rows
        $affectedRows = $statement->rowCount();

        if($affectedRows != 0)
        {
            return true;
        } else {
            return false;
        }
    } catch (PDOException $e) {
        die("ERROR: Could not execute query. " . $e->getMessage());
    }
}
```



PROGRAMMAZIONE WEB E SERVIZI DIGITALI

PHP E STRUMENTI DI PERSISTENZA (MySQL)

